

Aprendizaje Automático (Machine Learning)

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá



24 de marzo de 2026

Aprendizaje no supervisado.

- ▶ Su objetivo es descubrir patrones o estructuras ocultas en los datos.
- ▶ El modelo trabaja solo con datos de entrada (X), sin etiquetas conocidas.

Ejemplos:

- ▶ Agrupamiento (clustering): segmentar clientes según su comportamiento.
- ▶ Reducción de dimensionalidad: simplificar los datos manteniendo la información esencial.

Agrupamiento (Clustering)

Estas técnicas buscan agrupar los datos en **clusters** según similitud:

- ▶ **K-Means:** Particiona los datos en K clusters minimizando la distancia al centroide.
- ▶ **Hierarchical Clustering:** Construye un árbol de clusters (dendrograma) fusionando o dividiendo clusters.
- ▶ **DBSCAN:** Agrupa datos densos y detecta outliers automáticamente.
- ▶ **Gaussian Mixture Models (GMM):** Modelo probabilístico donde cada cluster es una distribución gaussiana; útil para estimar probabilidades de pertenencia a cada cluster.

Nota: Uso similar a clasificación, ya que se asigna cada dato a un cluster como si fuera una “clase” no etiquetada.

K-Means: Agrupamiento de Datos

- ▶ K-Means es un algoritmo de **clustering** que agrupa datos en K clústeres.
- ▶ Cada clúster está representado por su **centroide**, el promedio de los datos que contiene.
- ▶ El algoritmo minimiza la **distancia intra-clúster**, buscando que los datos dentro de un mismo clúster sean lo más similares posible.

Paso 1: Inicialización

- ▶ **Seleccionar K centroides iniciales aleatorios:**

$$\mu_1, \mu_2, \dots, \mu_K$$

- ▶ Cada centroide representa temporalmente un clúster.
- ▶ Esta selección puede afectar la convergencia y el resultado final.

Paso 2: Asignación de Clústeres

- ▶ Para cada dato x_i , **calcular la distancia a cada centroide.**
- ▶ Asignar x_i al clúster cuyo centroide esté más cercano:

$$\text{clúster}(x_i) = \arg \min_j \|x_i - \mu_j\|^2$$

- ▶ Esto forma K grupos basados en la proximidad a los centroides.

Paso 3: Actualización de Centroides

- ▶ Para cada clúster C_j , **recalcular el centroide** como promedio de los datos asignados:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

- ▶ Esto ajusta la posición del centroide hacia el “centro de masa” de sus datos.

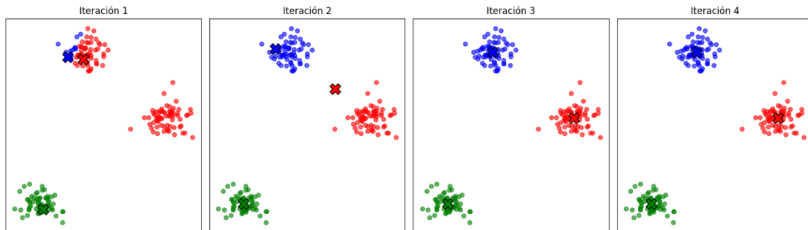
Paso 4: Repetición hasta convergencia

- ▶ Repetir los pasos de **asignación** y **actualización** hasta que:
 - ▶ Los centroides ya no cambien significativamente, o
 - ▶ Se alcance un número máximo de iteraciones.
- ▶ La función objetivo minimizada es la suma de distancias cuadradas dentro de cada clúster:

$$J = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Ejemplo Visual de K-Means

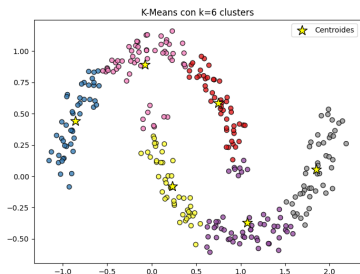
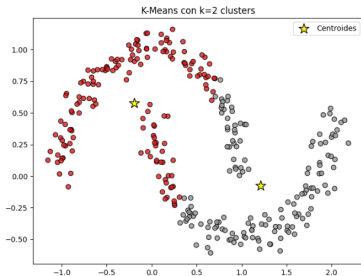
Evolución de K-Means: centroides y clústeres



- ▶ Los datos se agrupan en K clústeres.
- ▶ Cada color representa un clúster diferente.
- ▶ Los centroides se muestran como estrellas o círculos más grandes.

Otro ejemplo.

- ▶ Conjunto de datos no lineal.
- ▶ Resultados con diferentes valores de K .



Ver código python.

Ventajas y Limitaciones de K-Means

▶ **Ventajas:**

- ▶ Simple y rápido.
- ▶ Escalable a grandes conjuntos de datos.
- ▶ Buen desempeño cuando los clústeres son esféricos y similares en tamaño.

▶ **Limitaciones:**

- ▶ Requiere definir K previamente.
- ▶ Sensible a centroides iniciales y a outliers.
- ▶ Puede fallar con clústeres de forma irregular.

Hierarchical Clustering: Agrupamiento Jerárquico

- ▶ Es un método de **clustering jerárquico** que construye una jerarquía de clústeres.
- ▶ No requiere definir el número de clústeres K previamente.
- ▶ Se puede representar mediante un **dendrograma**, mostrando cómo se fusionan o dividen los clústeres.

Tipos de Clustering Jerárquico

► Aglomerativo (Bottom-Up):

- Cada datos comienza como un clúster individual.
- Se fusionan los clústeres más cercanos iterativamente.

► Divisivo (Top-Down):

- No es muy común.
- Todos los datos comienzan en un único clúster.
- El cluster se divide en 2 subclusters, por ejemplo usando k -medias.
- Los subclusteres se dividen iterativamente en dos subclústeres hasta alcanzar un criterio:
 - obtener k clústers o
 - que ya no tenga sentido dividir

Clustering Jerárquico Aglomerativo

Paso 1: Definir la distancia entre datos

- ▶ Se calcula una matriz de distancias entre todos los datos:

$$D = \begin{bmatrix} 0 & d_{12} & \dots & d_{1N} \\ d_{21} & 0 & \dots & d_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ d_{N1} & d_{N2} & \dots & 0 \end{bmatrix}$$

- ▶ Distancia común: Euclidiana

$$d_{ij} = \sqrt{\sum_{k=1}^M (x_{ik} - x_{jk})^2}$$

Paso 2: Fusión de clústeres

- ▶ Se seleccionan los clústeres más cercanos y se fusionan.
- ▶ **Distancias entre clústeres:**

- ▶ **Single linkage (distancia mínima):**

$$d_{\min}(A, B) = \min_{x \in A, y \in B} \|x - y\|$$

- ▶ **Complete linkage (distancia máxima):**

$$d_{\max}(A, B) = \max_{x \in A, y \in B} \|x - y\|$$

- ▶ **Average linkage (distancia promedio):**

$$d_{\text{avg}}(A, B) = \frac{1}{|A||B|} \sum_{x \in A} \sum_{y \in B} \|x - y\|$$

donde $|Z|$ es el número de elementos (datos) en el clúster Z .

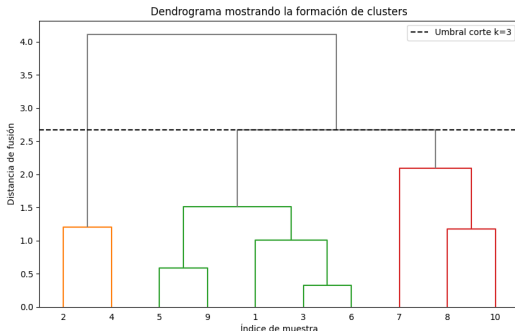
- ▶ **Ward (minimiza la varianza):**

$$\Delta E = \frac{|A||B|}{|A| + |B|} \|\bar{x}_A - \bar{x}_B\|^2$$

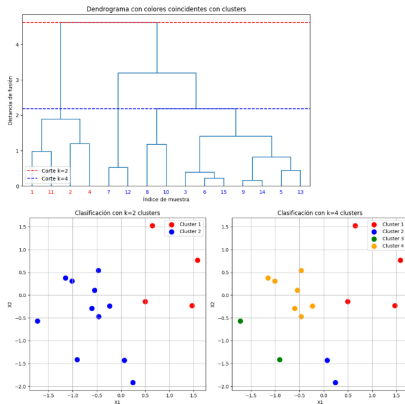
donde $\bar{x}_A$ y $\bar{x}_B$ son los centroides de los clústeres A y B .

Paso 3: Construcción del Dendrograma

- ▶ Cada fusión de clústeres se representa como un nodo en el dendrograma.
- ▶ El eje vertical indica la distancia o disimilitud entre los clústeres fusionados.
- ▶ Permite elegir el número de clústeres cortando el dendrograma a cierta altura.

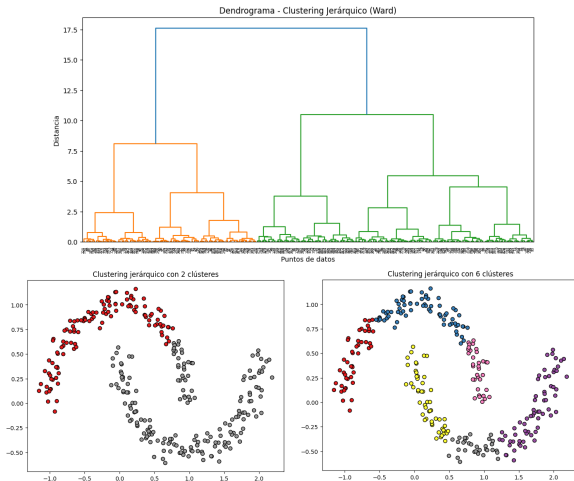


Ejemplo Visual de Clustering Jerárquico



- ▶ Los datos se agrupan de forma jerárquica.
- ▶ Diferentes colores representan los clústeres resultantes al cortar el dendrograma.

Ejemplo. con datos no lineales.



Ver código python.

Ventajas y Limitaciones del Clustering Jerárquico.

▶ Ventajas:

- ▶ No requiere especificar K previamente.
- ▶ Produce una estructura jerárquica interpretativa.
- ▶ Flexible en la elección de medidas de distancia.

▶ Limitaciones:

- ▶ Computacionalmente **costoso para grandes conjuntos de datos**.
- ▶ Sensible a ruido y outliers.
- ▶ Una vez fusionados/divididos, los clústeres no se pueden deshacer.

DBSCAN

- ▶ **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) es un algoritmo basado en densidad.
- ▶ Agrupa datos que se encuentran en regiones densas y marca como *outliers* los datos en regiones dispersas.
- ▶ No requiere especificar el número de clústeres previamente.

Parámetros principales:

- ▶ ε (radio): define la vecindad de un dato.
- ▶ `minPts`: número mínimo de datos necesarios para formar una región densa.

Definiciones:

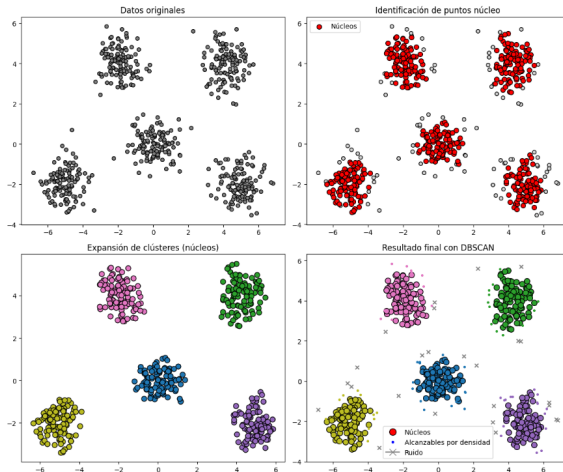
- ▶ Un dato es **núcleo** si tiene al menos `minPts` dentro de su ε -vecindad.
- ▶ Un dato es **alcanzable por densidad** si está dentro de la ε -vecindad de un dato núcleo.
- ▶ datos que no son núcleo ni alcanzables se consideran **ruido**.

Proceso de DBSCAN:

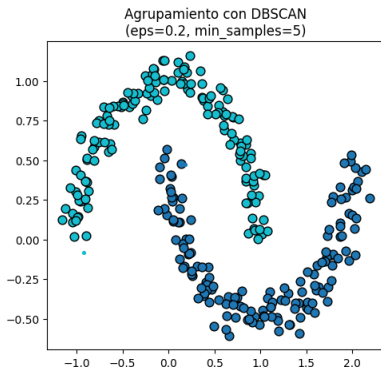
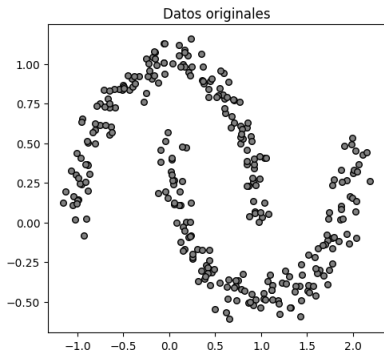
1. Seleccionar un dato no visitado.
2. Determinar su ε -vecindad.
3. Si tiene al menos `minPts` vecinos, se inicia un nuevo clúster.
4. Expandir el clúster incorporando todos los datos alcanzables por densidad.
5. Repetir hasta procesar todos los datos.

Ejemplo visual.

- Dataset bidimensional con cinco regiones densas y datos dispersos.



Ejemplo. con datos no lineales



Ver código python.

Ventajas y Limitaciones del Clustering DBSCAN

▶ Ventajas:

- ▶ Detecta clústeres de forma arbitraria (no solo esféricos).
- ▶ Maneja *outliers* de manera natural.
- ▶ No requiere especificar k (número de clústeres).

▶ Limitaciones:

- ▶ Sensible a la elección de ε y minPts .
- ▶ **Dificultad en datasets con densidades muy variables.**

Gaussian Mixture Models (GMM)

- ▶ Un **Gaussian Mixture Model (GMM)** asume que los datos provienen de una combinación de varias distribuciones gaussianas.
- ▶ Cada clúster se modela mediante una gaussiana con media μ_k y covarianza Σ_k .
- ▶ A diferencia de K-Means, los datos tienen una **probabilidad de pertenecer** a cada clúster.

Modelo probabilístico:

- ▶ La densidad conjunta se expresa como una mezcla ponderada de gaussianas:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

donde:

- ▶ π_k : peso del clúster k , con $\sum_k \pi_k = 1$.
- ▶ $\mathcal{N}(x \mid \mu_k, \Sigma_k)$: distribución normal multivariada.

Estimación con Expectation-Maximization (EM):

1. **Inicialización:** Asignar valores iniciales a μ_k, Σ_k, π_k .
2. **Paso E (Expectation):** Calcular responsabilidades

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}.$$

3. **Paso M (Maximization):** Recalcular parámetros usando γ_{ik} .

$$N_k = \sum_{i=1}^N \gamma_{ik}, \quad \mu_k = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} x_i, \quad (N: \text{núm. muestras.})$$

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top, \quad \pi_k = \frac{N_k}{N}.$$

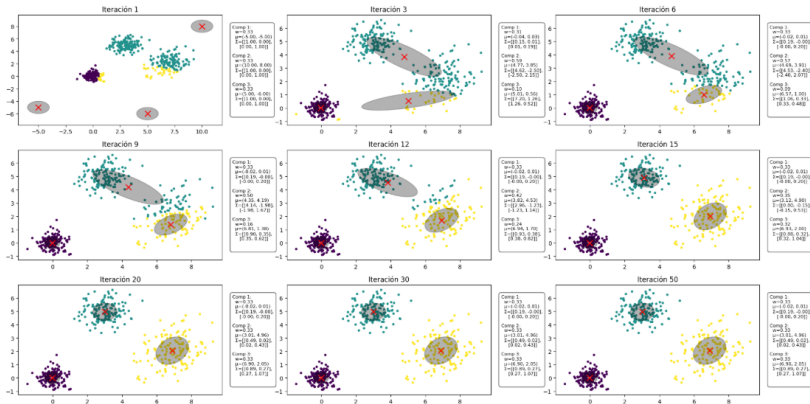
4. **Repetir** hasta convergencia

Nota práctica:

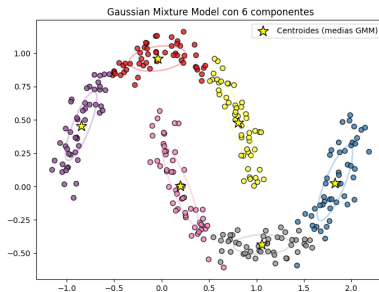
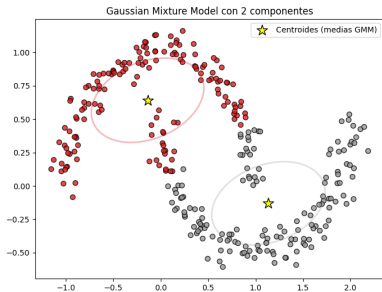
- ▶ Si Σ_k es diag (GMM esférico/diagonal), las fórmulas se aplican elemento a elemento; para estabilidad, puede usarse $\Sigma_k \leftarrow \Sigma_k + \varepsilon I$.
- ▶ ε representa un valor muy pequeño y positivo.

Ejemplo ilustrativo.

- Dataset bidimensional con tres clústeres.



Otro ejemplo.



Ver código python.

Ventajas y Limitaciones del GMM

▶ Ventajas:

- ▶ **Captura clústeres de forma elíptica**, no solo esférica.
- ▶ Ofrece pertenencia probabilística a cada clúster.
- ▶ Generaliza K-Means (caso particular cuando $\Sigma_k = \sigma^2 I$).

▶ Limitaciones:

- ▶ Puede converger a óptimos locales.
- ▶ Requiere especificar K .
- ▶ Sensible a la inicialización y datos ruidosos.

Reducción de dimensionalidad

Estas técnicas proyectan los datos a un espacio de menor dimensión, útil para visualización, compresión o extracción de características:

- ▶ **PCA (Principal Component Analysis):** Encuentra combinaciones lineales que explican máxima varianza.
- ▶ **t-SNE / UMAP:** Proyecciones no lineales para visualización de alta dimensión.
- ▶ **Autoencoders:** Redes neuronales que aprenden una representación comprimida de los datos.

Nota: Uso similar a regresión, ya que se estima un valor continuo en un espacio latente (las componentes principales o codificaciones).

Análisis de Componentes Principales (PCA)

- ▶ **PCA** es una técnica de reducción de dimensionalidad.
- ▶ Identifica nuevas variables, llamadas **componentes principales**, que capturan la mayor varianza de los datos.
- ▶ Permite proyectar datos en un espacio de menor dimensión conservando la información relevante.

Objetivos:

- ▶ Reducir la dimensionalidad de los datos.
- ▶ **Retener la mayor cantidad de varianza** posible.
- ▶ Facilitar la visualización y análisis de datos.

PCA como una optimización

- ▶ El Análisis de Componentes Principales (PCA) busca direcciones en los datos que **maximicen la varianza**.
- ▶ Sea $X \in \mathbb{R}^{n \times d}$ la matriz de datos centrados, donde:
 - ▶ d es la dimensión de los datos (vectores de características) y
 - ▶ n es el número de datos.
- ▶ La matriz de covarianza es:

$$S = \frac{1}{n} X^T X$$

Definición del problema

- ▶ Se pretende encontrar un vector unitario $w \in \mathbb{R}^d$ que maximice la varianza de las proyecciones:

$$\max_w \text{Var}(Xw)$$

- ▶ Como los datos están centrados:

$$\text{Var}(Xw) = w^T S w$$

- ▶ Restricción:

$$\|w\|^2 = 1$$

Problema de optimización

$$\max_w w^T S w, \quad \text{sujeto a } \|w\|^2 = 1$$

- ▶ Este es un problema de **optimización cuadrática con restricciones**.
- ▶ Se resuelve con multiplicadores de Lagrange:

$$\mathcal{L}(w, \lambda) = w^T S w - \lambda(w^T w - 1)$$

Condición de optimalidad

- ▶ Derivando respecto a w :

$$\nabla_w \mathcal{L} = 2Sw - 2\lambda w = 0$$

- ▶ Lo cual implica:

$$Sw = \lambda w$$

- ▶ Es decir:

- ▶ w debe ser un **autovector** de S y
- ▶ λ su autovalor asociado.

Interpretación

- ▶ El vector w que maximiza la varianza es el **autovector principal** de S .
- ▶ La **varianza máxima** es:

$$\text{Var}(Xw) = \lambda_{\text{máx}}$$

- ▶ Los siguientes componentes principales corresponden a los autovectores asociados a los autovalores decrecientes.

PCA = problema de optimización que maximiza la varianza proyectada.

Resumen:

- ▶ Dado un conjunto de datos centrado $X \in \mathbb{R}^{n \times d}$, se busca una proyección $Y = XW$, donde:

$$W = [w_1, w_2, \dots, w_m], \quad m \leq d$$

- ▶ Cada vector w_i es una dirección que maximiza la varianza proyectada:

$$w_i = \arg \max_{\|w\|=1} \text{Var}(Xw)$$

- ▶ Los vectores w_i son los **autovectores** de la matriz de covarianza de X .

Proceso paso a paso:

1. **Centrar los datos:** Restar la media de cada variable.
2. **Calcular la matriz de covarianza:**

$$\Sigma = \frac{1}{n-1} X^T X$$

3. **Obtener autovalores y autovectores** de Σ .
4. **Seleccionar los principales componentes** con mayor autovalor.
5. **Proyectar los datos** en el subespacio definido por los componentes seleccionados.

Paso 1: Centrar los datos

- ▶ Sea el conjunto de datos $X \in \mathbb{R}^{n \times d}$, con n muestras y d variables.
- ▶ Calcular la media de cada variable:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

- ▶ **Restar la media a cada observación** para centrar los datos:

$$X_{\text{centrado}} = X - \mathbf{1}\mu^T$$

- ▶ Esto garantiza que cada columna tenga media cero, lo cual es necesario para PCA.

Paso 1 de PCA: centrar los datos.

Dado el conjunto de datos:

$$X^T = \begin{bmatrix} 2.5 & 0.5 & 2.2 & 1.9 & 3.1 & 2.3 & 2.0 & 1.0 & 1.5 & 1.1 \\ 2.4 & 0.7 & 2.9 & 2.2 & 3.0 & 2.7 & 1.6 & 1.1 & 1.6 & 0.9 \end{bmatrix}$$

Calcular la media de cada columna:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \end{bmatrix} = \begin{bmatrix} 1.81 \\ 1.91 \end{bmatrix}$$

Centrar los datos restando la media:

$$X_{\text{centrado}}^T = \begin{bmatrix} 0.69 & -1.31 & 0.39 & 0.09 & 1.29 & 0.49 & 0.19 & -0.81 & -0.31 & -0.71 \\ 0.49 & -1.21 & 0.99 & 0.29 & 1.09 & 0.79 & -0.31 & -0.81 & -0.31 & -1.01 \end{bmatrix}$$

Los datos centrados se utilizan como entrada para el cálculo de la matriz de covarianza en el siguiente paso de PCA.

Paso 2: Calcular la matriz de covarianza

- ▶ La matriz de covarianza describe cómo varían conjuntamente las variables:

$$\Sigma = \frac{1}{n-1} X_{\text{centrado}}^T X_{\text{centrado}} \in \mathbb{R}^{d \times d}$$

$$X_{\text{centrado}} \in \mathbb{R}^{n \times d}$$

$$X_{\text{centrado}}^T X_{\text{centrado}} = \begin{bmatrix} \sum_{i=1}^n x_{i1}^2 & \sum_{i=1}^n x_{i1}x_{i2} & \cdots & \sum_{i=1}^n x_{i1}x_{id} \\ \sum_{i=1}^n x_{i2}x_{i1} & \sum_{i=1}^n x_{i2}^2 & \cdots & \sum_{i=1}^n x_{i2}x_{id} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^n x_{id}x_{i1} & \sum_{i=1}^n x_{id}x_{i2} & \cdots & \sum_{i=1}^n x_{id}^2 \end{bmatrix}$$

- ▶ Elemento (i, j) de Σ :

$$\sigma_{ij} = \frac{1}{n-1} \sum_{k=1}^n (x_{ki} - \mu_i)(x_{kj} - \mu_j)$$

- ▶ Σ es simétrica y positiva semidefinida, **garantizando autovalores reales y no negativos.**

Paso 2 de PCA: cálculo de la matriz de covarianza.

Dados los datos centrados $X_{\text{centrado}} \in \mathbb{R}^{10 \times 2}$, la matriz de covarianza se calcula como:

Los elementos de Σ pueden expresarse como:

$$\sigma_{11} = \frac{1}{n-1} \sum_{i=1}^n (x_{i1} - \bar{x}_1)^2,$$

$$\sigma_{12} = \sigma_{21} = \frac{1}{n-1} \sum_{i=1}^n (x_{i1} - \bar{x}_1)(x_{i2} - \bar{x}_2),$$

$$\sigma_{22} = \frac{1}{n-1} \sum_{i=1}^n (x_{i2} - \bar{x}_2)^2.$$

Para estos datos centrados se obtienen las sumas:

$$\sum_{i=1}^{10} (x_{i1} - \bar{x}_1)^2 = 5.549, \quad \sum_{i=1}^{10} (x_{i1} - \bar{x}_1)(x_{i2} - \bar{x}_2) = 5.539,$$

$$\sum_{i=1}^{10} (x_{i2} - \bar{x}_2)^2 = 6.449.$$

Dividiendo por $n - 1 = 9$ se obtiene la matriz de covarianza:

$$\Sigma = \frac{1}{9} \begin{bmatrix} 5.549 & 5.539 \\ 5.539 & 6.449 \end{bmatrix} = \begin{bmatrix} 0.61655556 & 0.61544444 \\ 0.61544444 & 0.71655556 \end{bmatrix}.$$

Esta Σ es la que se usa en el paso siguiente de PCA para obtener autovalores y autovectores.

Paso 3: Obtener autovalores y autovectores

- ▶ Resolver la ecuación característica de Σ :

$$\Sigma v = \lambda v$$

- ▶ Donde:
 - ▶ $v \in \mathbb{R}^d$ es un autovector (dirección de máxima varianza)
 - ▶ $\lambda \in \mathbb{R}$ es el autovalor correspondiente (varianza explicada)
- ▶ Esto produce un conjunto de pares (λ_i, v_i) para $i = 1, \dots, d$

Paso 3 de PCA: Cálculo de los autovalores y autovectores.

Los autovalores y autovectores de Σ se calculan resolviendo:

$$\det(\Sigma - \lambda I) = 0$$

y

$$\Sigma v = \lambda v$$

- ▶ **Valores propios (autovalores):**

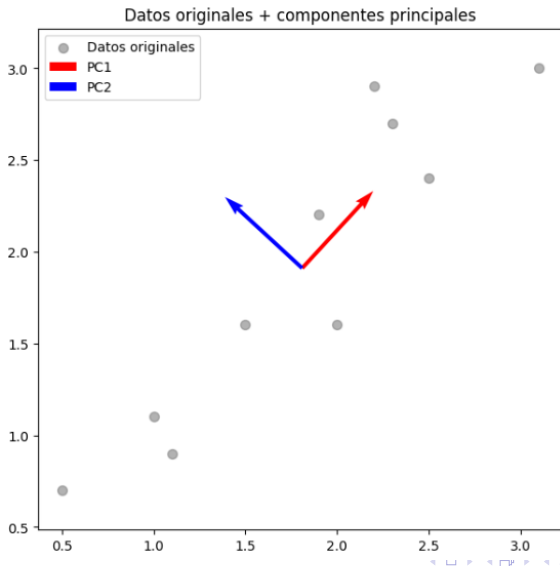
$$\lambda_1 \approx 1.284, \quad \lambda_2 \approx 0.049$$

- ▶ **Vectores propios (autovectores):**

$$v_1 \approx \begin{bmatrix} 0.6779 \\ 0.7352 \end{bmatrix}, \quad v_2 \approx \begin{bmatrix} -0.7352 \\ 0.6779 \end{bmatrix}$$

- ▶ Estos vectores determinan las direcciones de los **componentes principales**.

Autovectores.



Paso 4: Seleccionar los principales componentes

- ▶ Ordenar los autovalores $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$
- ▶ Seleccionar los k autovectores asociados a los mayores autovalores:

$$V_k = [v_1 \ v_2 \ \dots \ v_k] \in \mathbb{R}^{d \times k}$$

- ▶ Estos definen el **subespacio de menor dimensión que conserva la mayor varianza.**
- ▶ Fracción de varianza explicada:

$$\text{Varianza explicada} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

Paso 4 de PCA: Seleccionar los principales componentes.

- ▶ Selección de $k = 1$ componente principal (mayor autovalor):

$$V_1 = [v_1] = \begin{bmatrix} 0.6779 \\ 0.7352 \end{bmatrix} \in \mathbb{R}^{2 \times 1}$$

- ▶ Esta dirección define el subespacio de menor dimensión que conserva la mayor parte de la varianza.
- ▶ **Fracción de varianza explicada:**

$$\text{Varianza explicada} = \frac{\lambda_1}{\lambda_1 + \lambda_2} \approx \frac{1.284}{1.284 + 0.049} \approx 0.963$$

Paso 5: Proyectar los datos

- ▶ Proyectar los datos centrados en el subespacio de componentes principales:

$$Z = X_{\text{centrado}} V_k \in \mathbb{R}^{n \times k}$$

- ▶ Cada fila de Z es la representación de x_i en el espacio reducido.
- ▶ Esta proyección **maximiza la varianza preservada y reduce la dimensionalidad.**

Paso 5 de PCA: Proyectar los datos.

- ▶ Dado el primer componente principal v_1 , la proyección de los datos centrados X_{centrado} se calcula como:

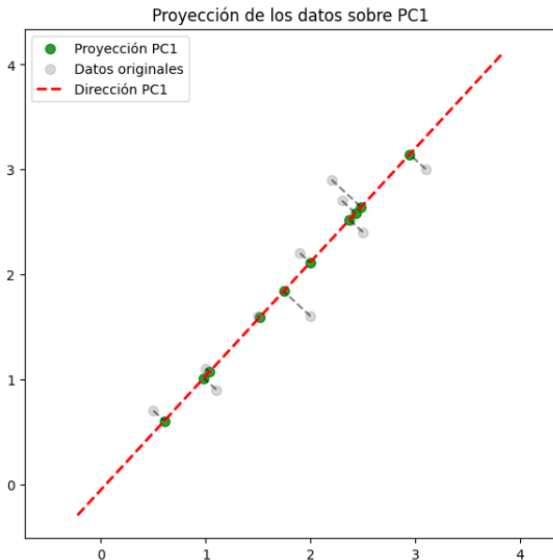
$$Z = X_{\text{centrado}} V_1 \in \mathbb{R}^{n \times 1}$$

- ▶ Cada elemento z_i de Z representa la coordenada de x_i en la dirección de máxima varianza.
- ▶ Ejemplo de proyección de los datos:

$$Z^T \approx \begin{bmatrix} 0.827 & -1.777 & 0.992 & 0.274 & 1.675 & 0.912 \\ & & -0.099 & -1.144 & -0.438 & -1.223 \end{bmatrix}$$

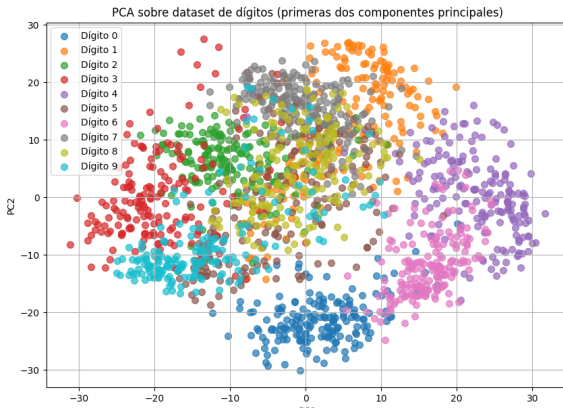
- ▶ Esta proyección reduce la dimensionalidad de 2D a 1D conservando aproximadamente el 96.3% de la varianza.
- ▶ Permite representar los datos en un espacio más compacto y visualizar la estructura principal.

Proyección de los datos a menor dimensión.



Ejemplo.

- ▶ Tamaño original de los datos: (1797, 64)
- ▶ Tamaño después de PCA: (1797, 29)
- ▶ Número de componentes seleccionadas: 29
- ▶ Varianza explicada acumulada: 0.9547965245651595



Ventajas y Limitaciones de PCA

▶ Ventajas:

- ▶ Reduce dimensionalidad y ruido.
- ▶ Facilita visualización de datos de alta dimensión.
- ▶ Conserva la mayor parte de la varianza original.

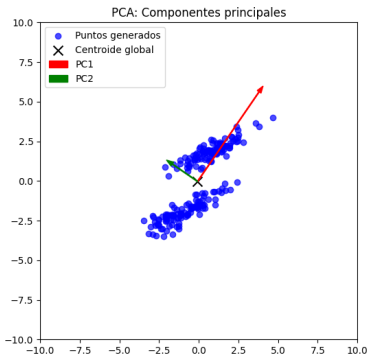
▶ Limitaciones:

- ▶ Los componentes principales no siempre son interpretables.
- ▶ Supone relaciones lineales entre variables.
- ▶ Sensible a la escala de las variables (recomendable normalizar).

Ver código python.

Limitación de PCA en clasificación

- ▶ PCA maximiza la varianza global de los datos, sin tener en cuenta las etiquetas de clase (**no supervisado**).
- ▶ Esto implica que las direcciones principales no necesariamente **maximizan la separabilidad entre clases**.
- ▶ Consecuencia: PCA puede proyectar datos de diferentes clases en direcciones que los mezclan.



LDA (Linear Discriminant Analysis)

- ▶ En clasificación, el objetivo es encontrar una proyección que **separe mejor las clases**.
- ▶ El Análisis Discriminante Lineal (LDA) incorpora la información de las etiquetas de clase.
- ▶ LDA es por tanto una **técnica supervisada**, a diferencia de PCA.

Idea: proyectar los datos en un espacio de baja dimensión donde las clases estén mejor separadas.

Definición matemática de LDA

- ▶ LDA busca un vector w tal que la proyección $y = w^T x$ maximice la separación entre clases.
- ▶ Se definen dos matrices de dispersión:
 - ▶ Dispersión **entre** clases:

$$S_B = \sum_{c=1}^C n_c (\mu_c - \mu)(\mu_c - \mu)^T$$

- ▶ Dispersión **intra**-clase:

$$S_W = \sum_{c=1}^C \sum_{x \in c} (x - \mu_c)(x - \mu_c)^T$$

Donde:

- ▶ C = número de clases en el problema.
- ▶ μ_c = media de los vectores de la clase c .
- ▶ μ = media global de todos los datos.
- ▶ n_c = número de muestras en la clase c .

Problema de optimización en LDA

- ▶ Criterio de Fisher

$$J(w) = \frac{w^T S_B w}{w^T S_W w}$$

- ▶ Se desea:

$$\max_w J(w)$$

- ▶ Intuición:

- ▶ Numerador grande $\Rightarrow$ clases bien separadas.
- ▶ Denominador pequeño $\Rightarrow$ poca dispersión dentro de cada clase.

Solución de LDA

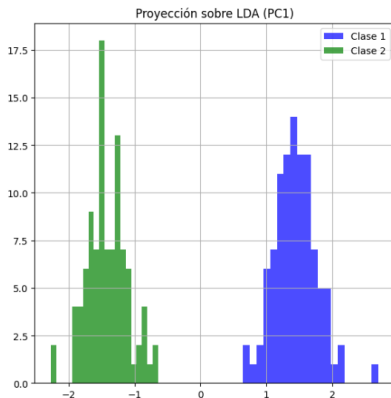
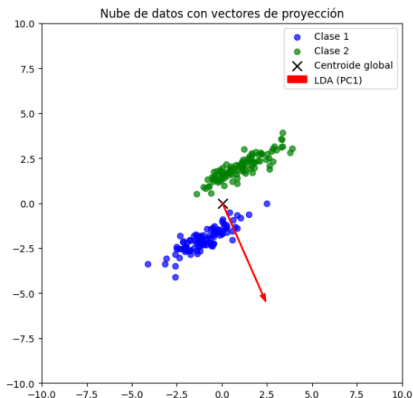
- ▶ La optimización lleva a un problema de valores propios:

$$S_W^{-1} S_B w = \lambda w$$

- ▶ Los vectores discriminantes son los autovectores asociados a los mayores autovalores de esta ecuación.
- ▶ En un problema con C clases, se pueden obtener hasta $C - 1$ componentes discriminantes.

LDA = proyección supervisada que maximiza la separabilidad entre clases.

Ejemplo: LDA.



Ver código python.

t-SNE (t-Distributed Stochastic Neighbor Embedding)

- ▶ t-SNE es una técnica de reducción de dimensionalidad no lineal.
- ▶ Su objetivo es **representar datos de alta dimensión en 2D o 3D** preservando la estructura local.
- ▶ **Mantiene las relaciones de proximidad:** datos cercanos en el espacio original permanecen cercanos en el espacio reducido.
- ▶ Se utiliza frecuentemente para **visualización de datos complejos** como imágenes, texto o genómica.

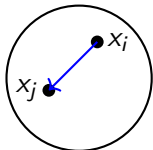
Principio de funcionamiento de t-SNE:

1. Calcular probabilidades de similitud entre datos en el espacio original.
2. Definir probabilidades en el espacio reducido con distribución t de Student.
3. Minimizar la función de costo divergencia de Kullback-Leibler entre P y Q .
4. Optimización mediante gradiente descendente.

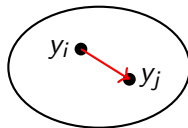
Intuición de t-SNE

- ▶ El objetivo es preservar la **estructura local**.
- ▶ Dos datos cercanos en el espacio original $\mathbb{R}^n$ deben mantenerse cercanos en la proyección 2D o 3D.
- ▶ Se comparan distribuciones de probabilidad en ambos espacios.

Espacio original $\mathbb{R}^n$



Espacio reducido $\mathbb{R}^2$



Paso 1: Cálculo de probabilidades en el espacio original (alta dimensión).

- ▶ La similitud condicional se define como:

$$p_{j|i} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}$$

- ▶ σ_i controla la **anchura de la vecindad** de cada dato.
- ▶ Se ajusta σ_i de manera que la **perplejidad**¹ de la distribución P_i coincida con un valor deseado.

¹perplejidad $\approx$ número de vecinos cercanos.

Medida de equilibrio entre vecinos cercanos y lejanos:

- ▶ La perplejidad (**hiperparámetro**) se define como:

$$\text{Perp}(P_i) = 2^{H(P_i)} \sim \text{Perplexity deseada}$$

donde

$$H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$$

- ▶ Intuitivamente, la perplejidad mide el número efectivo de vecinos considerados para el dato i .
- ▶ Valores típicos: entre 5 y 50.

Ajuste automático.

- ▶ Para cada dato i , se busca un σ_i tal que:

$$\text{Perp}(P_i) \approx \text{Perplejidad deseada}$$

- ▶ El **procedimiento**:

1. Inicializar un rango para σ_i (por ejemplo $[10^{-5}, 10^5]$).
2. Calcular la perplejidad resultante con un valor de prueba de σ_i .
3. Ajustar el rango superior o inferior según si la perplejidad es mayor o menor que la deseada.
4. Repetir hasta convergencia (búsqueda binaria).

Paso 2: Distribución en el espacio embebido (baja dimensión):

- ▶ En lugar de una gaussiana (como en PCA), se utiliza una distribución de Student- t de una sola cola.
- ▶ La probabilidad de similitud en el espacio reducido se define como:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

- ▶ Esta elección permite **una mejor separación de los clusters en el espacio reducido** *atenúa el crowding problem*², gracias a que la Student- t posee colas más pesadas, lo cual permite separar mejor los clusters en el espacio reducido.

²En alta dimensión, cada dato puede tener muchos vecinos cercanos.

Al proyectar en baja dimensión, **no hay suficiente “espacio”** para ubicar a todos esos vecinos cercanos de forma proporcional.

Como resultado, los datos tienden a **amontonarse en el centro** del mapa, dificultando la correcta representación de las distancias relativas.

Paso 3: Función de costo (KL Divergence):

- ▶ El objetivo es que q_{ij} reproduzca lo mejor posible a p_{ij} .
- ▶ Es decir, minimizar la divergencia de Kullback-Leibler (KL) entre la distribución de similitudes en el espacio original (P) y la distribución en el espacio de baja dimensión (Q):

$$C = KL(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- ▶ Esto equivale a maximizar la similitud local:
 - ▶ Si dos datos son vecinos en alta dimensión (p_{ij} grande), se empujan en baja dimensión para que también tengan q_{ij} grande.

Paso 4: Optimización (Descenso de gradiente):

- ▶ El costo C se minimiza mediante *gradient descent*.
- ▶ La derivada es:

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$

- ▶ El factor 4 es solo una conveniencia de escala.
- ▶ El término $(p_{ij} - q_{ij})$ determina la dirección:
 - ▶ Si $p_{ij} > q_{ij}$: los datos se acercan.
 - ▶ Si $p_{ij} < q_{ij}$: los datos se alejan.

Hiperparámetros en t-SNE

- ▶ **Perplexity ($\mathcal{P}$)**: mide el número efectivo de vecinos.

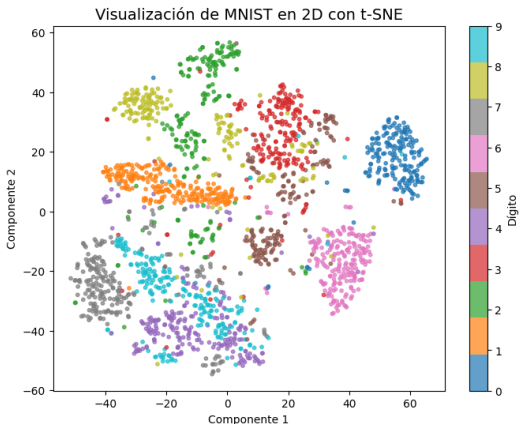
$$\mathcal{P} = 2^{H(P_i)} \quad \text{donde } H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$$

Valores típicos: entre 5 y 50.

- ▶ **Número de iteraciones**: controla la convergencia del gradiente.
- ▶ **Learning rate**: define la magnitud de los pasos en la optimización.
- ▶ **Semilla aleatoria**: asegura reproducibilidad de los resultados.

Ejemplo: Visualización con t-SNE (MNIST)

- ▶ Dataset: dígitos escritos a mano con 784 dimensiones.
- ▶ Objetivo: proyectar a 2D manteniendo agrupaciones de dígitos similares.



Distribuciones de Clusters en t-SNE

- ▶ La técnica **t-SNE** es un método de reducción de dimensionalidad **no supervisado**.
- ▶ Para obtener distribuciones de clusters, es necesario aplicar:
 - ▶ Un algoritmo de **clustering** en el espacio original.
 - ▶ El mismo algoritmo en el espacio reducido.

Espacio de Alta Dimensión

- ▶ Se tiene un conjunto de datos: $X \in \mathbb{R}^d$
- ▶ Se aplica un algoritmo de clustering (ej. **K-Means**) con K clusters.
- ▶ Cada muestra x_i es asignada a: $\hat{c}_{\text{original}}(i)$
- ▶ La distribución se calcula como frecuencia relativa:

$$P_{\text{original}}(k) = \frac{\#\{i \mid \hat{c}_{\text{original}}(i) = k\}}{N}$$

donde: N es el número total de muestras y $k \in \{1, \dots, K\}$ es el índice del cluster

Clustering en el Espacio Reducido

- ▶ Después de t-SNE se obtiene una representación reducida:

$$Y \in \mathbb{R}^2$$

- ▶ Se aplica nuevamente clustering:
 - ▶ Mismo algoritmo (ej. K-Means)
 - ▶ Mismo número de clusters K
- ▶ Se obtienen nuevas etiquetas:

$$\hat{c}_{\text{reducido}}(i)$$

- ▶ La nueva distribución es:

$$P_{\text{reducido}}(k) = \frac{\#\{i \mid \hat{c}_{\text{reducido}}(i) = k\}}{N}$$

- ▶ Permite analizar cómo t-SNE reorganiza los datos.

Comparación de Distribuciones

- ▶ Se comparan:

$$P_{\text{original}} \quad \text{vs} \quad P_{\text{reducido}}$$

- ▶ Consideraciones importantes:
 - ▶ Los índices de cluster no son directamente comparables.
 - ▶ K-Means puede asignar etiquetas diferentes en cada espacio.
 - ▶ Se analiza la **distribución global**.
- ▶ Para comparar clusters directamente se puede:
 - ▶ Alinear clusters entre espacios
 - ▶ Usar métodos de asignación óptima

Ejemplo: Visualización con t-SNE (MNIST)

- Comparación de distribuciones usando k-means para agrupar.

Distribución de probabilidad por cluster (comparativa):

Cluster	Probabilidad antes de t-SNE	Probabilidad después de t-SNE
0	0.1175	0.1080
1	0.0955	0.1025
2	0.0855	0.0675
3	0.1640	0.1430
4	0.0930	0.1060
5	0.1030	0.0865
6	0.0845	0.1130
7	0.0645	0.0950
8	0.1180	0.1165
9	0.0745	0.0620

Ventajas y Limitaciones de t-SNE

▶ Ventajas:

- ▶ Excelente para visualización de datos de alta dimensión.
- ▶ Captura estructura local y relaciones de proximidad.
- ▶ Funciona bien con clusters complejos y no lineales.

▶ Limitaciones:

- ▶ No preserva necesariamente la estructura global.
- ▶ Computacionalmente costoso para datasets muy grandes.
- ▶ Resultados sensibles a hiperparámetros como *perplexity* y número de iteraciones.

Ver código python.

UMAP (Uniform Manifold Approximation and Projection)

- ▶ UMAP es otra técnica de reducción de dimensionalidad no lineal.
- ▶ Construye un grafo de vecinos más cercanos en el espacio original.
- ▶ Optimiza una representación en el espacio reducido preservando la estructura topológica y la distancia relativa.
- ▶ Más rápido y escalable que t-SNE en datasets grandes.

Principio de funcionamiento de UMAP.

1. Construcción del grafo de vecinos k -NN ponderado según similitud local.
2. Representación estocástica de los vecinos mediante probabilidades de conexión.
3. Optimización de una función de pérdida que preserva la estructura del grafo en el espacio reducido.
4. Proyección final en 2D o 3D para visualización o análisis.

Paso 1: Construcción del grafo k -NN.

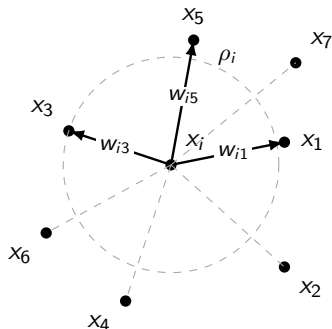
- ▶ Se calcula la distancia entre cada dato $x_i \in \mathbb{R}^D$ y sus k vecinos más cercanos.
- ▶ Se asigna un peso a la conexión usando:

$$w_{ij} = \exp\left(-\frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}\right)$$

donde:

- ▶ $d(x_i, x_j)$: distancia euclidiana entre datos.
 - ▶ ρ_i : distancia mínima no nula de x_i a un vecino.
 - ▶ σ_i : parámetro de normalización local.
- ▶ Resultado: un grafo ponderado dirigido que captura la **estructura local**.

Construcción del grafo k -NN ($k = 3$ vecinos)



- ▶ ρ_i es la distancia mínima no nula desde x_i a un vecino,
- ▶ Los pesos w_{ij} inducen probabilidades p_{ij} .

Paso 2: Representación probabilística.

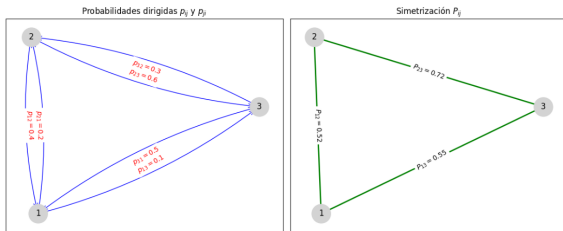
- Se interpreta cada peso como una probabilidad de conexión:

$$p_{ij} = w_{ij}$$

- Para obtener un grafo no dirigido, se simetriza:

$$P_{ij} = p_{ij} + p_{ji} - p_{ij}p_{ji}$$

- Esto garantiza una **distribución estocástica de similitud**.



$$P_{23} = 0.6 + 0.3 - 0.3 \cdot 0.6 = 0.72$$

Paso 3: Función de pérdida.

- ▶ Se define un grafo análogo en el espacio reducido (2D o 3D) con probabilidades de conexión que decrecen con la distancia:

$$Q_{ij} = \frac{1}{1 + a\|y_i - y_j\|^{2b}}$$

- ▶ $a, b > 0$ son parámetros ajustados automáticamente para que la función $f(d) = 1/(1 + ad^{2b})$ se aproxime lo mejor posible a la curva de conectividad deseada, en función del parámetro de *mínima distancia* y de la escala de los datos.
 - ▶ b : Controla la curvatura (qué tan rápido cae la probabilidad).
 - ▶ a : actúa como un factor de escala para que el rango de distancias corresponda con el grafo original.
- ▶ La función de pérdida busca minimizar la divergencia de **entropía cruzada binaria** entre P_{ij} y Q_{ij} :

$$\mathcal{L} = \sum_{i \neq j} \left[p_{ij} \log \frac{p_{ij}}{q_{ij}} + (1 - p_{ij}) \log \frac{1 - p_{ij}}{1 - q_{ij}} \right]$$

Paso 4: Proyección en espacio reducido.

- ▶ Los datos $y_i \in \mathbb{R}^d$ (con $d = 2$ o 3) se **optimizan** para minimizar $\mathcal{L}$.
- ▶ Se utiliza **descenso estocástico de gradiente** con técnicas de muestreo negativo.

Cálculo del Gradiente de la pérdida (par a par):

- ▶ Sea la contribución al coste de un par (i, j) :

$$\mathcal{L}_{ij} = - [P_{ij} \log Q_{ij} + (1 - P_{ij}) \log(1 - Q_{ij})].$$

- ▶ Los pasos principales para calcular la derivada respecto a y_i :
 - ▶ Derivada de $\mathcal{L}$ respecto a Q_{ij} :

$$\frac{\partial \mathcal{L}_{ij}}{\partial Q_{ij}} = \frac{Q_{ij} - P_{ij}}{Q_{ij}(1 - Q_{ij})}.$$

- Derivada de Q_{ij} respecto a y_i .
Sea $r = \|y_i - y_j\|$, entonces

$$Q_{ij} = \frac{1}{1 + ar^{2b}} \Rightarrow \frac{\partial Q_{ij}}{\partial y_i} = -2ab r^{2b-2} Q_{ij}^2 (y_i - y_j).$$

- Por la regla de la cadena:

$$\frac{\partial \mathcal{L}_{ij}}{\partial y_i} = \frac{Q_{ij} - P_{ij}}{Q_{ij}(1 - Q_{ij})} \cdot \frac{\partial Q_{ij}}{\partial y_i}.$$

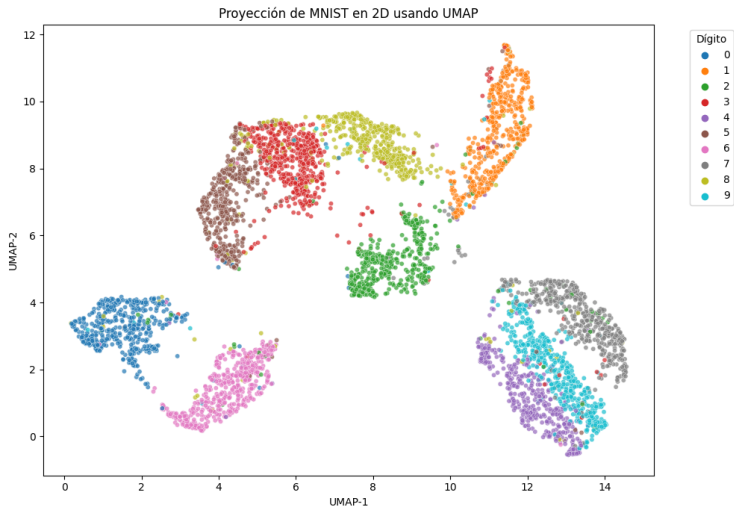
- Simplificando (usando $Q_{ij}/(1 - Q_{ij}) = 1/(ar^{2b})$) se obtiene una forma compacta del gradiente

$$\frac{\partial \mathcal{L}_{ij}}{\partial y_i} = -2b(Q_{ij} - P_{ij}) \frac{(y_i - y_j)}{r^2},$$

válida para $r \neq 0$ (en la práctica se añade un ε al denominador).

- La derivada respecto a y_j es $\frac{\partial \mathcal{L}_{ij}}{\partial y_j} = -\frac{\partial \mathcal{L}_{ij}}{\partial y_i}$, por simetría.
- Si $Q_{ij} > P_{ij}$: la fuerza es de **repulsión**.
- Si $Q_{ij} < P_{ij}$: la fuerza es de **atracción**.

Ejemplo: Visualización con UMAP (MNIST)



Ventajas y Limitaciones de UMAP.

► Ventajas:

- Escalable y rápido en grandes datasets.
- Preserva tanto estructura local como global mejor que t-SNE.
- Permite reducir dimensionalidad a cualquier número de dimensiones.

► Limitaciones:

- Sensible a la selección de hiperparámetros (*n_neighbors*, *min_dist*).
- Interpretación matemática más compleja que PCA.

Ver código python.

Comparación de las técnicas de reducción de dimensionalidad.

▶ PCA:

- ▶ Método lineal basado en varianza.
- ▶ Preserva estructura global pero no captura bien relaciones no lineales.

▶ t-SNE:

- ▶ No lineal, enfocado en la estructura local.
- ▶ Ideal para clusters complejos.

▶ UMAP:

- ▶ Similar a t-SNE pero más rápido y con mejor preservación global.
- ▶ Basado en teoría de grafos y topología algebraica.

Autoencoders

- ▶ Los **autoencoders** son redes neuronales que aprenden una representación comprimida de los datos de entrada.
- ▶ Objetivo: reconstruir la entrada a partir de un espacio latente de menor dimensión.
- ▶ Utilizados para reducción de dimensionalidad, eliminación de ruido y detección de anomalías.

Arquitectura de un Autoencoder.

- ▶ Compuesto por dos partes principales:

1. **Encoder:** transforma la entrada $x \in \mathbb{R}^d$ en un vector latente $z \in \mathbb{R}^k$, donde $k < d$:

$$z = f_{\theta}(x)$$

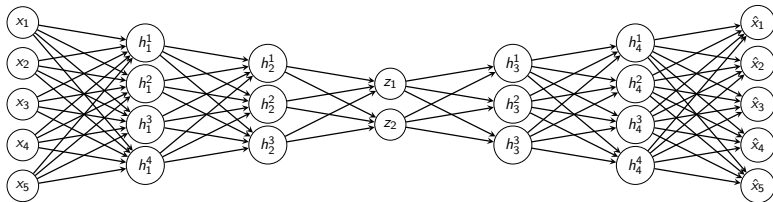
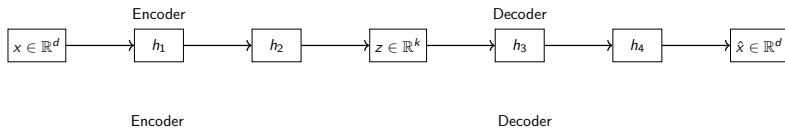
2. **Decoder:** reconstruye la entrada desde z :

$$\hat{x} = g_{\phi}(z)$$

- ▶ La red se entrena minimizando la función de pérdida de reconstrucción:

$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Arquitectura de un Autoencoder.



Proceso de entrenamiento.

1. Inicializar los pesos de encoder y decoder.
2. Propagación hacia adelante:

$$z = f_{\theta}(x), \quad \hat{x} = g_{\phi}(z)$$

3. Calcular la pérdida de reconstrucción:

$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

4. Propagación hacia atrás (backpropagation) para actualizar los pesos.
 5. Repetir hasta que la pérdida converja.
- La red puede ser **fully connected**, convolucional o recurrente, según el tipo de datos.

- ▶ Una vez entrenado, el encoder permite mapear datos a un espacio latente z de menor dimensión:

$$z_i = f_{\theta}(x_i)$$

- ▶ Este espacio latente puede usarse para:
 - ▶ Visualización de datos complejos en 2D/3D.
 - ▶ Clustering o clasificación usando z_i como características.
 - ▶ Detección de anomalías: datos con reconstrucción pobre tienen alta pérdida.

- ▶ El encoder entrenado permite mapear datos de alta dimensión $x_i \in \mathbb{R}^d$ a un espacio latente $z_i \in \mathbb{R}^k$ de menor dimensión ($k \ll d$):

$$z_i = f_{\theta}(x_i)$$

- ▶ **Interpretación del espacio latente:**

- ▶ Cada dato z_i representa una versión comprimida y relevante de x_i .
- ▶ Las relaciones de proximidad en x_i se preservan parcialmente en z_i , dependiendo de la arquitectura y pérdida usada.

- ▶ **Usos prácticos:**

- ▶ **Visualización:** proyectar z_i en 2D o 3D para observar clusters o patrones.
- ▶ **Clustering o clasificación:** usar z_i como nuevas características de menor dimensión para algoritmos como k-means, SVM o Random Forest.
- ▶ **Detección de anomalías:** comparar la reconstrucción $\hat{x}_i = g_{\phi}(z_i)$ con la entrada x_i ; errores grandes indican datos atípicos.

Ventajas y Limitaciones.

► Ventajas:

- Captura **relaciones no lineales complejas** en los datos.
- Permite reducción de dimensionalidad más flexible que PCA lineal.
- Puede detectar anomalías y eliminar ruido.
- **Ventajas de usar z_i para reducción de dimensionalidad:**
 - Mantiene información relevante mientras reduce ruido.
 - Facilita visualización y análisis exploratorio.
 - Permite acelerar otros algoritmos al trabajar con menos dimensiones.

► Limitaciones:

- Requiere entrenamiento de red neuronal y ajuste de hiperparámetros.
- Riesgo de sobreajuste si la red es muy profunda.
- La interpretación de la representación latente puede ser compleja.

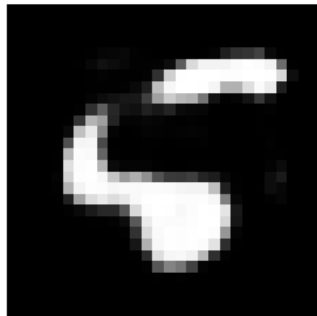
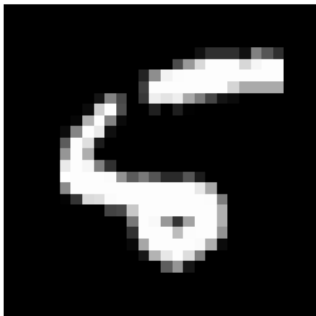
Ejemplo ilustrativo.

- ▶ Dataset: imágenes de dígitos (MNIST) de 28x28 píxeles, es decir, 748 características.
- ▶ Encoder: reduce la imagen a un vector latente de 64 dimensiones.
- ▶ Decoder: reconstruye la imagen desde el vector latente.
- ▶ Resultado:
 - ▶ Entrada: imagen original de dígito "5".
 - ▶ Salida: imagen reconstruida, similar al original.
 - ▶ Representación latente captura características relevantes del dígito.
- ▶ Los autoencoders permiten visualizar, generar y analizar los datos en un espacio reducido.

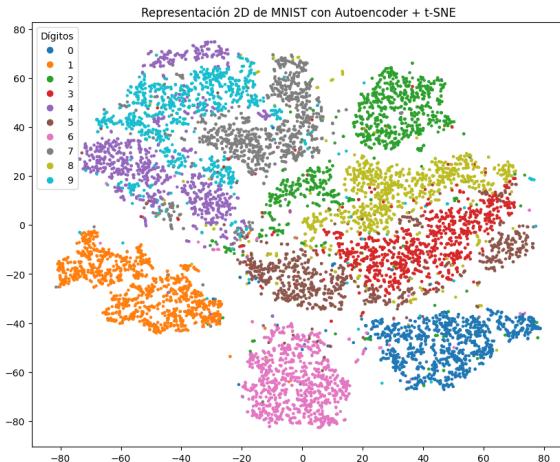
Ejemplo ilustrativo.

- ▶ Forma original: (60000, 784)
- ▶ Forma reducida: (60000, 64)

Comparación de un dígito '5' original vs reconstruido



Ejemplo ilustrativo.



Ver código python.

Evaluación en Aprendizaje No Supervisado.

- ▶ A diferencia del aprendizaje supervisado, no siempre existen etiquetas de referencia.
- ▶ La evaluación se basa en:
 - ▶ **Métricas internas:** analizan la estructura de los datos.
 - ▶ **Métricas externas:** comparan con etiquetas, si están disponibles.
 - ▶ **Estabilidad:** verificar si los resultados son consistentes.

Evaluación de Clustering.

Métricas internas.

- ▶ No requieren de etiquetas verdaderas.
- ▶ **Inercia / SSE:** mide la compacidad de los clústeres.

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

- ▶ **Coeficiente de Silhouette:** evalúa qué tan bien separado está un clúster de los demás.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

donde $a(i)$ es la distancia promedio al clúster propio y $b(i)$ al más cercano.

- ▶ **Índice de Davies-Bouldin (DBI):** compara dispersión intra-clúster con separación inter-clúster.

- ▶ Sea k el número de clústeres:

$$S_i = \frac{1}{|C_i|} \sum_{x \in C_i} \|x - c_i\| \quad (\text{dispersión intra-clúster})$$

$$M_{ij} = \|c_i - c_j\| \quad (\text{distancia entre centroides})$$

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{S_i + S_j}{M_{ij}} \right)$$

- ▶ Valores bajos de DBI indican clústeres compactos y bien separados.

Métricas externas.

Se usan cuando existen etiquetas verdaderas.

► **Rand Index (RI) y Adjusted Rand Index (ARI)**

- Miden similitud entre agrupamientos.
- Considere un conjunto de n elementos agrupados en dos particiones:
 - a : número de pares en el mismo clúster en ambas particiones.
 - b : número de pares en clústeres diferentes en ambas particiones.
 - c : número de pares en el mismo clúster en la partición 1 pero en distinto clúster en la partición 2.
 - d : número de pares en clústeres diferentes en la partición 1 pero en el mismo clúster en la partición 2.

► **Rand Index (RI):** $RI = \frac{a+b}{a+b+c+d}$

► **Adjusted Rand Index (ARI):**

$$ARI = \frac{\binom{n}{2}(a+b) - [(a+c)(a+d) + (b+c)(b+d)]}{\frac{1}{2}[(a+c)(a+d) + (b+c)(b+d)] - [(a+c)(a+d) + (b+c)(b+d)]}$$

- $RI \in [0, 1]$ y $ARI \in [-1, 1]$. Valores cercanos a 1 indican buen ajuste.

► Normalized Mutual Information (NMI):

Evalúa la información compartida entre clústeres y etiquetas.

- Sea $\Omega = \{C_1, C_2, \dots, C_K\}$ la partición real y $\mathcal{C} = \{C'_1, C'_2, \dots, C'_L\}$ la partición obtenida.
- La **Información Mutua** entre ambas particiones se define como:

$$I(\Omega, \mathcal{C}) = \sum_{i=1}^K \sum_{j=1}^L \frac{|C_i \cap C'_j|}{n} \log \left(\frac{n \cdot |C_i \cap C'_j|}{|C_i| \cdot |C'_j|} \right)$$

- La **Entropía** de cada partición es:

$$H(\Omega) = - \sum_{i=1}^K \frac{|C_i|}{n} \log \left(\frac{|C_i|}{n} \right), \quad H(\mathcal{C}) = - \sum_{j=1}^L \frac{|C'_j|}{n} \log \left(\frac{|C'_j|}{n} \right)$$

- La **NMI** se define como:

$$NMI(\Omega, \mathcal{C}) = \frac{I(\Omega, \mathcal{C})}{\sqrt{H(\Omega) \cdot H(\mathcal{C})}}$$

- ▶ **Exactitud de clustering:** cuando es posible mapear clústeres a clases.
 - ▶ Mide el grado de correspondencia entre las etiquetas verdaderas y las etiquetas predichas por el algoritmo de clustering.
 - ▶ Sea $\{y_1, y_2, \dots, y_n\}$ el conjunto de etiquetas verdaderas y $\{c_1, c_2, \dots, c_n\}$ las etiquetas asignadas por el clustering.
 - ▶ Dado que la asignación de etiquetas en clustering no es única, se busca la mejor permutación π de las clases predichas.
 - ▶ Matemáticamente se define como:

$$ACC = \max_{\pi \in \mathcal{P}} \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i = \pi(c_i)\}$$

donde:

- ▶ $\mathcal{P}$ es el conjunto de todas las permutaciones posibles de etiquetas.
- ▶ $\mathbf{1}\{\cdot\}$ es la función indicadora (vale 1 si la condición es verdadera y 0 en caso contrario).
- ▶ Valores de ACC cercanos a 1 indican un clustering más preciso.

Evaluación de Reducción de Dimensionalidad.

► Preservación de distancias:

- Mide qué tan bien se mantienen las distancias entre datos después de aplicar una reducción de dimensionalidad.
- Sea $X = \{x_1, x_2, \dots, x_n\}$ el conjunto original de datos y $Y = \{y_1, y_2, \dots, y_n\}$ su representación reducida.
- Se define el **coeficiente de preservación de distancias** como la correlación entre las matrices de distancia:

$$DP = \text{corr}(D_X, D_Y)$$

donde:

- $D_X(i, j) = \|x_i - x_j\|$ es la distancia original entre x_i y x_j ,
- $D_Y(i, j) = \|y_i - y_j\|$ es la distancia en el espacio reducido.
- Alternativamente, se puede medir mediante el error cuadrático medio normalizado:

$$DP = 1 - \frac{\sum_{i < j} (D_X(i, j) - D_Y(i, j))^2}{\sum_{i < j} D_X(i, j)^2}$$

- DP cercano a 1 indica una alta preservación de distancias.

► Trustworthiness:

- Mide qué tan fielmente la reducción de dimensionalidad preserva las relaciones de vecindad locales.
- Sea $X = \{x_1, \dots, x_n\}$ el conjunto de datos original y $Y = \{y_1, \dots, y_n\}$ su representación reducida.
- Sea $U_k(i)$ el conjunto de datos que están entre los k vecinos más cercanos de i en Y , pero no en X .
- Se define como:

$$T(k) = 1 - \frac{2}{nk(2n - 3k - 1)} \sum_{i=1}^n \sum_{j \in U_k(i)} (r_X(i, j) - k),$$

donde:

- $r_X(i, j)$ es el **rango** (posición) de j respecto a i en el espacio original X .
- n es el número de muestras.
- k es el número de vecinos considerados.
- Valores cercanos a 1 indican alta confianza en la preservación de la vecindad.

► Continuity:

- Mide en qué medida los vecinos en el espacio original se mantienen en la representación reducida.
- Sea $X = \{x_1, \dots, x_n\}$ el conjunto de datos original y $Y = \{y_1, \dots, y_n\}$ su representación reducida.
- Sea $V_k(i)$ el conjunto de datos que están entre los k vecinos más cercanos de i en X , pero no en Y .
- Se define como:

$$C(k) = 1 - \frac{2}{nk(2n - 3k - 1)} \sum_{i=1}^n \sum_{j \in V_k(i)} (r_Y(i, j) - k),$$

donde:

- $r_Y(i, j)$ es el **rango** (posición) de j respecto a i en el espacio reducido Y .
- n es el número de muestras.
- k es el número de vecinos considerados.
- Valores cercanos a 1 indican que los vecinos del espacio original se preservan en la representación reducida.

Otros enfoques.

Visualización: en 2D/3D permite evaluar separabilidad de grupos.

Detección de anomalías:

- ▶ Con etiquetas: métricas clásicas (precisión, recall, F1).
- ▶ Sin etiquetas: validación con expertos o inspección de anomalías.

Estabilidad:

- ▶ Reentrenar el modelo con distintas semillas o subconjuntos.
- ▶ Verificar consistencia en resultados (clústeres, embeddings, anomalías).

Gracias por su atención.

